

Landscape of High-Performance Python to Develop Data Science and Machine Learning Applications

OSCAR CASTRO, PIERRICK BRUNEAU, and JEAN-SÉBASTIEN SOTTET, Luxembourg Institute of Science and Technology, Luxembourg
DARIO TORREGROSSA, Goodyear Innovation Center, Luxembourg

Python has become the prime language for application development in the data science and machine learning domains. However, data scientists are not necessarily experienced programmers. Although Python lets them quickly implement their algorithms, when moving at scale, computation efficiency becomes inevitable. Thus, harnessing high-performance devices such as multi-core processors and graphical processing units to their potential is generally not trivial. The present narrative survey can be thought of as a reference document for such practitioners to help them make their way in the wealth of tools and techniques available for the Python language. Our document revolves around user scenarios, which are meant to cover most situations they may face. We believe that this document may also be of practical use to tool developers, who may use our work to identify potential lacks in existing tools and help them motivate their contributions.

CCS Concepts: • **Computing methodologies** → **Machine learning**; **Parallel programming languages**; • **Software and its engineering** → **Software development techniques;**

Additional Key Words and Phrases: Python, code acceleration, data science

ACM Reference format:
Oscar Castro, Pierrick Bruneau, Jean-Sébastien Sottet, and Dario Torregrossa. 2023. Landscape of High-Performance Python to Develop Data Science and Machine Learning Applications. *ACM Comput. Surv.* 56, 3, Article 65 (October 2023), 30 pages.
https://doi.org/10.1145/3617588

1 INTRODUCTION

Python is one of the most used computer programming languages today: it was ranked in the first position on the PYPL (PopularitY of Programming Language) index [65] and the first position on the TIOBE index [83] in 2022.

Python is intensively used in the growing domains of **Data Science (DS)**, scientific computation, data analytics, and **Machine Learning (ML)**. It is used as the successor of the many data-centric and scientific computation programming languages, such as R, Fortran, and Matlab. One of the main reasons behind this success in DS stands on its many DS- and ML-focused libraries, such as

O. Castro is now with the University of Luxembourg.
Authors' addresses: O. Castro, P. Bruneau, and J.-S. Sottet, Luxembourg Institute of Science and Technology, 5 avenue des Hauts Fourneaux, Esch/Alzette, Luxembourg; e-mails: {oscar.castro, pierrick.bruneau, jean-sebastien.sottet}@list.lu; D. Torregrossa, Goodyear Innovation Center, Avenue Gordon Smith, Colmar-Berg, Luxembourg; e-mail: dario_torregrossa@goodyear.com.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2023 Copyright held by the owner/author(s).
0360-0300/2023/10-ART65
https://doi.org/10.1145/3617588

NumPy, Pandas, TensorFlow, Scikit-learn, SciPy, and Matplotlib. Given the amount of data being collected and processed within the DS and ML contexts, most Python high-performance libraries have been developed outside Python by using statically typed languages such as C++, Fortran, and/or CUDA.

The main reasons that explain the fact that libraries are developed outside Python are the slow performances of the Python interpreter. Ismail and Suh [39] studied in detail the overheads coming with Python code execution. First, as is generally true of interpreted languages, it is slower than running compiled code. Indeed, like most interpreted languages (e.g., Java), Python programs are translated to bytecode before execution by a virtual machine. An additional inherent inefficiency comes with Python due to its dynamic object typing system. Importantly, Ismail and Suh [39] identify that C function calls from the interpreter, invoked when calling a compiled library function, yield significant overheads. A disposable execution environment then must be set up and cleaned, which brings a constant per-instruction overhead.

The default implementation, CPython, uses the **Global Interpreter Lock (GIL)**. The GIL offers some safety mechanisms for concurrent accesses, and in return it prevents multi-threading: the interpreter executes only a single thread within a single CPython process. The aim of the GIL is to simplify the implementation by making the object model safe against concurrent access. This means that CPython executes CPU-bound code in a single thread. In addition, evaluations showed that CPython exhibits poor instruction-level parallelism in this context [39]. As a result, in terms of efficiency, Python is not doing well if compared with other languages. Therefore, there are different tools available to improve the performance of programs built in Python.

The objective of this review article is to provide an organized landscape of Python high-performance tools. We can relate to surveys considering the acceleration and optimization of communications in the context of high-performance computing [69], or improving compilation to accelerate code execution [7]. Parallelization of inputs and outputs in the context of high-performance computing [11], software optimization in view of energy efficiency of embedded systems [59], and **Graphical Processing Unit (GPU)** acceleration in the context of a specific ML task (e.g., Frequent Itemset Mining [15]) are also closely related domains. But to our knowledge, we propose the first survey of high-performance and code acceleration tools restricted to the Python programming language. An originality of our approach is also to organize our work according to user scenarios. As such, it aims at identifying the different categories of approaches used for Python code acceleration regarding different prototypical DS and ML practitioner profiles. This will facilitate adoption by practitioners looking for guidance while providing principal axes for disclosing our results. In Section 2, we introduce our method and approach regarding this survey, notably our specific viewpoint based on practitioner profiles and scenarios. Specifically, we motivate the definition of three user profiles, designed to cover most situations and needs faced in the practice of DS and ML. We then disclose the details of the associated search strategy. In Sections 3, 4 and 5, each mirroring the respective user profile, we group Python tools, primarily according to their relevance to the identified profiles, then by the concepts and techniques involved in view to improve Python performance. As this may not lead to a strict taxonomy, a versatile tool that may be applicable in more than one profile will be described in the section most closely matching its main usage scenario, as reflected in *quick-start* sections and tutorials commonly seen for the tool. References will be used from other sections as needed. Besides describing the tools and providing quantitative and qualitative information about them, care is taken to emphasize similarities, complementarities, and potential conflicts between tools and approaches. In Section 6, we summarize our findings and delineate some important lessons learned. We conclude the article in Section 7.

2 METHOD

This article presents a narrative review covering the domain of the acceleration of Python program execution. Thus, it aims at providing practitioners with an overview of the current state of the art in high-performance Python programming, notably through parallelization, distributed execution, and code transformation.

This review was initially motivated by our practice of Python programming in the domains of DS and ML, where improving performance is critical to obtain timely results [17]. This initial work led us to realize that a systematic review focused on high-performance Python with the practice of DS and ML in mind is lacking in the current literature. As our work is driven by pragmatic concerns, we believe that a narrative review is the most suitable format.

2.1 Approach and Context

As with any narrative review, we will do a qualitative evaluation of the diverse extant approaches in the domain of performance improvement of Python programs. To lead this survey, the authors relied on their expertise, in the related fields, including industrial experience and research communities: ML, DS, and software engineering. Their different backgrounds and institutions, as well as years of experience in the aforementioned fields, helped considering both academical and practical point of view, in a way as broad and relevant as possible.

However, the landscape of tools and techniques in this scope is quite diverse, and may be distinguished according to a large number of facets (e.g., level of automation, close ties with a peculiar DS task, expected amount of effort to put in use). As a result, there is no obvious hierarchy in these facets that would drive the structure of the taxonomy presented in this article. Instead, we focus on three common usage scenarios, which are meant to be mostly mutually exclusive, while covering most situations met in practice by data scientists:

- The developed algorithm may involve some non-standard data structure, such as a special kind of knowledge graph. To avoid the effort of searching for an existing library that can be adapted to fit the requirements, a pure prototypical Python algorithm may have been developed by a data scientist to solve a theoretical or practical problem. However, upon validation, the algorithm's performance significantly degrades when applied to larger datasets. As a result, there is a need to explore alternative approaches for more efficient computations.
- Most commonly, DS practitioners face situations close to canonical problem involving standard data structures such as numerical matrices or graphs. Practitioners will then design an algorithm to solve the problem and implement it using popular numerical Python libraries such as NumPy or Pandas. After validating the algorithm, they need to apply it to larger data sets, but runtime becomes excessive.
- Finally, the algorithm may still be only on paper, and instead of boldly starting implementing it in vanilla Python, the data scientist may look for the right library or framework to directly maximize computational efficiency at implementation time, even if it involves learning to master a Domain Specific Language (DSL) or non-standard constructs.

In these three scenarios, performance is sought, but from differing starting points, and with variable will to invest in mastering sophisticated tools. For instance, in the two first scenarios, implementations are already developed, and the goal is to look for cost-effective solutions to scale them up. In contrast, the third scenario is bound to a longer-term view, where the practitioner will tolerate to invest in the most appropriate tool from the start.

Given this context, the objective of this survey is to answer the following question: which approaches can be used to improve Python execution performance in the context of one of these three scenarios?

Table 1. Search Plan Summary

Source	Criteria
Database	Web-based resources: Google Scholar, GitHub, PyPI, Reddit, Stack Overflow, Data Science Central
Date of publication	2018–2022 (for scientific publications)
Keywords	(*high-performance* or *parallelization* or *multicore* or *GPU* or *performance improvement* or *performance enhancement* or *acceleration* or *compilation* or *transformation*) combined with (*Python* and (*machine learning* or *data science*))
Language	English
Type of publication	Conference proceedings Journal articles PyPI documentation GitHub vignettes Social media posts Web news and articles
Inclusion criteria	Tools/Packages aimed at increasing the performance of Python code for ML and DS directly or indirectly
Exclusion criteria	Tools/packages outside Python Python packages focusing on one application use case Packages dedicated to a specific type of hardware other than CPU and GPU

2.2 Search Strategy

Our search strategy started by using Google Scholar, which is a search engine that indexes metadata or even the whole text of scientific publications in multiple scientific repositories. To reflect our inclusion and exclusion criteria (Table 1), we used the keyword *Python* combined with *data science* or *machine learning*, as well as other keywords associated with high-performance computation and code acceleration (see Table 1). The first seed of relevant papers thus collected could point us to other important works by inspecting the cited references. We also looked at implementations and repositories referred to in this set of selected papers. In addition, we covered social networks and forums commonly used by DS practitioners and software engineers for exchanging and sharing, such as Reddit, Stack Overflow, Kaggle, and Data Science Central. We applied our search strategy on reference code repositories and Python package indexes as well, mostly GitHub and PyPI (Python Package Index). Scientific publications were primarily filtered by date between 2018 and 2022. Nonetheless, we also reviewed some tools dated before that time frame if they had very high relevance to our scope or if they had been actively maintained. The summary of the search strategy is presented in Table 1.

As stated previously, we focus on performance improvement in the context of three identified profiles associated with the preceding defined scenarios. For the section scenario, in Section 4, we motivate the subset of libraries under our focus and describe how the search strategy was specifically amended in that case.

2.3 Inclusion and Exclusion Criteria

The selected publications and tools must propose some performance improvement of the execution of Python programs that directly impacts DS or ML tasks. Some tools with a wider scope may

be included, provided that they clearly enable to accelerate the execution of DS or ML tasks, or relevant function in tier DS or ML packages.

We restricted the resulting scope to the Python realm. We are aware that many high-performance libraries and research studies exist independently of Python, but our choice is motivated by the relative monopoly of Python in the domains of DS and ML. Some of the approaches, mainly developed in C/C++ but accessible thanks to wrappers in Python, are obviously considered. Although we focus on recent tools and contributions, we also report about seminal work if its lineage can be directly related to recent practice. A high number of works are from 2015 and later due to the relative acceleration of contributions in ML and DS recently. The diversity implied by our scenarios led us to consider all levels of granularity in terms of enhancing the performance of Python code: from very general code transformation approaches to numerical libraries widely used in the DS domain.

We focused on the default CPython interpreter and thus did not consider alternative Python interpreters (e.g., Pyston [48]) intentionally. We address this point in a specific paragraph in the discussion (Section 6.3). Besides pure code optimization, we will consider approaches exploiting multiple CPUs (and CPU cores), as well as GPUs, the latter being widely used in ML. Nevertheless, we did not dig into application-specific or dedicated processors like tensor processing units and field-programmable gate arrays.

3 PURE PYTHON PERFORMANCE IMPROVEMENT

In this section, we focus on tools and approaches that support acceleration of code in which the computationally intensive parts rely only on the default Python distribution (also called *vanilla Python*). In the context of our scenarios, this can be because the modeling of the problem at hand is not standard and thus not necessarily compliant with existing numerical computation libraries (e.g., custom knowledge graph). This can be also because the practitioner is more comfortable with vanilla Python for working on an implementation that sticks to some algorithmic formalism in the literature.

Acceleration approaches in this section will generally target Python at large, beyond traditional DS use cases. In this scenario, we thus assume that a working version of the code addressing the problem of the practitioner has already been developed. It works on small samples of data but now needs to be accelerated to be scaled up to larger amounts of data. DS and ML tasks often involve loops performing the same operation on many chunks of data (e.g., when independently processing images or database records). The seminal way to accelerate such code is to implement the *Single Instruction, Multiple Threads* (SIMT) principle using tools derived from **Message Passing Interface (MPI)** or **Open Multi-Processing (OpenMP)** libraries. Tools in Python related to this approach are presented in Section 3.1.

3.1 Distributed Memory and Shared Memory Approaches

MPI works with a distributed memory model, potentially exploiting a distributed network of machines. MPI is a message passing standard that defines the syntax and semantic of library routines to develop parallel applications. With MPI, computers running a parallel program can exchange messages. MPI was originally designed to develop programs in the languages C, C++, and Fortran. Nonetheless, some Python libraries offer the same bindings for MPI (e.g., MPI4Py [25] and PyPar [70]).

Conversely, OpenMP works on a shared memory model for multi-core CPUs using program directives. The OpenMP standard [24] provides a set of code annotations and instructions for the compiler and a runtime library that extends Fortran and C/C++ languages to express shared memory parallelism. OpenMP is based on compiler directives, and thus less intrusive in the code than

MPI (i.e., not requiring a strong refactoring of the existing codebase). Based on those directives, it allows the compiler to parallelize chunks of code whose instructions can be shared among the processors. OpenMP is supported by the most common compilers, such as Clang, LLVM, and GCC. It supports loop-level, nested, and task parallelism. Commonly, annotations or directives of the OpenMP API are used in loops. OpenMP has two main related implementations in Python; one of the most famous is Pymp [45]. It is a library that proposes a special language construction to behave like OpenMP. It relies on the system Fork mechanism instead of threads to make parallel computation. It tries to reduce its footprint by referencing memory and not copying everything in the forked process. The second one is PyOMP [46], which is based on Numba and offers a set of constructs similar to the OpenMP API. Nevertheless, the compilation pipeline for Python is a bit more complex: PyOMP uses Numba to generate code in LLVM, then machine code to be able to run it.

3.2 Task-Based Approaches

Task-based frameworks are more recent tools to parallelize and distribute heavy computation. Although they are generally powerful, their usage may imply heavy code refactoring. More complex frameworks, which impose a specific development approach, are detailed in Section 5.2. In this section, we cover libraries which rely on such frameworks but aim at parallelizing an existing piece of code with minimal impact using decorators. A decorator is an instruction set before the definition of a function. A decorator indicates that a function (associated with the decorator) must transform a user function (the decorated function) and extend the behavior of the latter function without explicitly making modifications.

Decorators are used to express parallelism by indicating that these functions are going to be treated as tasks. The decorated code is analyzed and converted (if applicable) into a suitable version for parallelization. Falling into this category, we found PyCOMPSs [82], Pygion [76] and PyKokkos [3], wrappers for COMPSs [81], Legion [9] and Kokkos [84], respectively. PyCOMPSs and Pygion share some similarities. Both libraries build a task dependency graph and perform analysis to define the order of task execution and the parallelism that can be achieved. Decorators are also similar, as PyCOMPSs and Pygion both use *@task*. However, PyKokkos translates Python code into the Kokkos API written in C++ and has more decorators to implement its programming model. In PyKokkos, for example, functions can be decorated with *@pk.workunit*. These functions can run in parallel by passing them as an argument to the function *parallel_for*. PyKokkos also has support for using GPUs with CUDA.

In Jug [20], a task is defined as a Python function, and its arguments take values or outputs of another task. Using the *@taskgenerator* decorator, Jug performs an analysis on a task dependency graph to define the execution order and parallelization of the tasks. Parallelization is achieved by running more than one Jug process for distributing the tasks and using synchronization to get a result. As it is developed with Python, libraries such as NumPy and Scikit learn are compatible with Jug.

Pydron is a library to parallelize sequential Python code through decorators [52]. Pydron targets multi-core, clusters, or cloud platforms. First, it translates the decorated functions in Python into an intermediate representation with a dataflow graph structure. The graph is analyzed by a scheduler that defines the order in which tasks are going to run by putting them in a queue, with some tasks being scheduled to run in parallel. When a task is finished, the scheduler must be informed, and based on the available information, it changes the execution graph. The tasks are distributed to be executed on worker nodes. There is a distribution system in charge of managing the hardware resources—commonly, a Python interpreter is launched per CPU core, each in charge to execute a given task.

3.3 Program Transformation and Compilation

Besides annotations, directives, and decorators for parallelization mentioned in previous sections, program transformation and compilation is another straightforward way to obtain better execution performance for an existing codebase with minimal work overhead for the practitioner. These approaches rely on code analysis that can be either static (source code) or dynamic (based on traced execution) before proposing a transformation of the code into a target language to obtain a better performance in their execution. As such, they can provide performance improvement in a general programming context.

The prominent approach we have found is to guide or give hints to the transformation tool, often a compiler, regarding specific sections of the code that should be optimized. These hints are expressed by the user by typing variables or adding decorators. Additionally, we review transformation tools which claim full automation, which means input code is passed as it is then.

3.3.1 Semi-Automatic Approaches. Cython, bundled with most Python distributions, is a language extension that serves as a superset of Python. It acts as a compiler, transforming high-level Cython code into highly efficient C code. One of the key advantages of Cython is its ability to write C extensions, seamlessly integrating them into Python programs for improved performance.[1] By translating Cython code into optimized C/C++ code and compiling it as Python extension modules, it unlocks the potential for faster execution compared to pure Python. Remarkably, the majority of Python code can be compiled by Cython without modifications, making it a straightforward tool for optimizing Python applications. To further enhance performance, it is crucial to add static type declarations to Cython code. These declarations enable the Cython compiler to generate simpler and faster C code. Additionally, Cython provides automatic conversions between Python objects and basic numeric and string types. Whereas Python handles memory allocation dynamically, Cython allows manual memory management similar to traditional C code.

Cython also provides parallelism mechanisms through the module *cython.parallel* using OpenMP as the backend [24]. To use the parallel module, the GIL must be released. When the GIL is released, Python objects cannot be manipulated. Therefore, a function that deals with Python objects cannot be directly invoked with parallel attributes: the data must be converted into Cython typed variables or memory views. Good candidates for Cython implementation are general mathematical operations, array operations, and loops. By just using static typing and replacing Python math operations, obtaining a speedup with Cython is highly probable, even if maximal gains require fairly good development skills. If not well exploited, the performance gain will only be marginal. Moreover, it requires a manual detection of the code parts that could really benefit from Cython, and it will depend on the ability of a programmer to use profilers to find out the bottlenecks of the execution of a program.

JIT stands for *Just-in-Time* compilation, a technique employed by some programming languages and runtime environments to enhance code execution performance. Python is an interpreted language, which means that code is executed directly by the interpreter without prior compilation. Although this approach offers flexibility and dynamic features, it can be slower compared to compiled languages. However, JIT compilation allows the interpreter to dynamically compile specific code sections into machine code just before execution. This runtime process optimizes the code for improved performance. Some alternative interpreters to CPython use JIT compilation to optimize their performance. As we left alternative interpreters outside of the main scope of our work, please refer to Section 6.3 for more information on this area.

A highly popular JIT compiler for Python is Numba [44]. Numba provides compilation of Python code for a faster execution. The user must use decorators to indicate code parts that should be

[1] https://cython.org/

improved by the compiler. A common function decorator in Numba is *@jit* and has the following parameters: *nopython*, *parallel*, and *fastmath*. If *nopython* is set to true, the JIT compiler would compile the decorated function, so it will try to run without the involvement of the Python interpreter. The *parallel* flag enables Numba with a transformation pass that will attempt to automatically parallelize and/or perform other optimizations on the function or some parts of it. The *fastmath* flag relaxes some numerical rigor to gain additional performance and enables possible fast-math optimizations. By executing the code, the Numba JIT would attempt to apply the improvements we indicated with the decorators and their parameters.

The Numba compiler translates Python code into an intermediate representation, then it is translated to LLVM to finally emit machine code. The generated machine code is close in terms of performance to a traditional compiled language. Similarly to Cython, Numba only supports a subset of the Python language and some specific libraries like NumPy. Numba can convert a sequential code to be executed in parallel by multiple cores and in very limited cases to be executed in a GPU. Numba can also be used as a bridge to develop programs in Python to run in the GPU. It offers support for CUDA (NVIDIA hardware), ROCm (AMD), and HSA (AMD and ARM). A big difference is that there are no automatic attempts to parallelize the code then. Instead, the user must refactor the code to a style similar to C with CUDA. Numba can compile a restricted subset of Python code into CUDA kernels and device functions, HSA kernels, and ROCm device functions. In GPU programming, a kernel is a GPU function launched by the host (CPU and its memory) and executed in parallel on the device (GPU and its memory). A device function is a GPU function executed on the device that can only be called from the device.

Designed within the context of astrophysical applications, Hope specializes in numerical computations. Hope is a JIT compiler that uses the decorator *@hope.jit* with the function to be translated. The decorated functions are parsed into a Python **Abstract Syntax Tree (AST)**. The Python AST is converted into a Hope AST. Several optimizations may be applied to the Hope AST, such as simplification of expressions, factorizing out subexpressions, and replacing the *pow* function for integer exponents. From the Hope AST, C++ code is generated and compiled into a shared library (*.so* file on Linux systems). The shared library is added to the cache, loaded, and executed. Hope validates the name of the functions and the types of the passed arguments and tries to match to what it has on the cache; if not found, then the whole compilation process starts over. The data types used in the functions are inferred by static analysis of code, the AST, and the runtime analysis. The simplification of expressions and common subexpression elimination is performed with the SymPy library [79].

Autoparallel [67] is a compiler for Python code to transform nested loops from sequential to parallel execution in a distributed computing infrastructure. It requires that the user adds a decorator on identified functions that contain nested loops. Autoparallel relies on PyCOMPSs [82] and PLUTO [12]; PyCOMPSs is a task-based programming model to develop applications with Python decorators (reviewed in Section 3.2), whereas PLUTO is a parallelization tool that automatically transforms affine loops using the polyhedral model [8]. Autoparallel analyzes code sections decorated with *@parallel*, and creates a *Scop* object for each affine nested loop found. The Scop object is then parallelized by adding OpenMP-like decorators to the loops. Then, it converts the code into task format through PyCOMPSs by adding tasks configurations and data synchronizations. Finally, each nested loop is replaced by the generated code to be executed by PyCOMPSs in a distributed computing platform.

3.3.2 Automatic Approaches. Transforming software in view to maximize performance is difficult to perform fully automatically. Code translation and transpilation focus on analyzing the structure of the code and apply transformation patterns as a means to circumvent the absence of supervision.

Due to the nature of Python as an interpreted language, an increase of performance can be obtained by just porting a Python program into a compiled language. Nonetheless, doing it manually is a cumbersome task. Therefore, some specialized libraries perform transpilation by translating Python code into a compiled language (most often C++).

Shed Skin uses static analysis by checking implicit types of variables. Therefore, Shed Skin requires that all variables are implicitly typed. In other words, they must only have one assignment, and multiple assignments of different types to the same variable is not supported. To use Shed Skin, a command must be used in a terminal and the file containing Python code is passed as an argument. The Shed Skin compiler generates the translated code in C++, a header file, and a make script to compile it. Moreover, a module can be compiled and invoked from another Python script.

Nuitka translates CPython instructions into a C++ program. Compiled code generated by Nuitka is executed along with the Python interpreter for the part that cannot be compiled. This means that compatibility with other libraries is supported while using Nuitka. No code modification is required. To use Nuitka, the code must be compiled using the console through Nuitka commands along with the Python code filename. The code and executable files are generated and can be invoked directly or as stand-alone libraries.

Pythran converts Python code into C++ code. However, it goes beyond pure translation and performs code analysis and optimizations. Pythran receives as an input a Python module meant to be converted into a shared library. On the frontend of Pythran, the Python module is converted into a Python AST. Then, the Python AST is converted into a Pythran internal representation that is a subset of the Python AST. During this conversion, code analysis steps and different transformations and optimizations are performed, aimed at generating a faster version of the code. Additionally, variable types may be inferred by static analysis. The backend of Pythran turns Pythran internal representation into parameterized C++ code. Then, Pythran instantiates and compiles the generated code to build a native module. Pythran is compatible with NumPy expressions and applies optimizations such as expression templates, loop vectorization, and loop parallelization through OpenMP.

Transpyle [16] relies on transpilation to accelerate Python performance. The originality of this approach is to support multiple languages also as input (e.g., reusing a legacy optimized loop written in Fortran and integrating it in the transpiled Python code). Moreover, with the use of Python as the intermediate representation for compiling code from and into target languages (e.g., Fortran), it helps the Python developer to understand the complete process. It also works in a semi-automated mode with Python annotations, possibly guiding the compiler for better improvements (e.g., loop unrolling and vectorization).

ALPyNA [40] is a program transformation tool for Python that uses static and dynamic analysis of nested loops and generates CUDA kernels for GPU execution. The input code must contain vanilla Python code and optionally NumPy instructions. Currently, basic subscripting of single or multi-dimensional arrays is supported (i.e., no slicing or sequence indexing). ALPyNA performs analysis mostly on nested loops, where a performance bottleneck is more likely to occur. Other Python instructions are ignored and are executed by the Python interpreter. After static analysis, if loop bounds and data dependencies can be determined, ALPyNA generates untyped GPU kernels. Otherwise, loops are marked for analysis at runtime. For runtime analysis (and execution), the ALPyNA execution object must be used (obtained by the function that performs static analysis) to invoke the original functions. If possible, loop bounds and data dependencies are determined at runtime and GPU kernels are generated on the fly. ALPyNA relies on Numba to finalize and compile the GPU kernels.

Pyjion is a JIT compiler designed to improve the performance of Python by converting CPython bytecode into machine code [63]. In the absence of Pyjion, CPython relies on a master evaluation

loop known as the frame evaluation loop to sequentially execute opcodes (individual instructions within the bytecode). However, Pyjion's compiler consists of three key stages. First, it constructs a *stack table* that maps abstract types to each opcode position. Then, it compiles CPython opcodes into CIL (Common Intermediate Language) opcodes. Finally, it emits these CIL opcodes to the .NET execution engine compiler, which converts them into native machine code or assembly. Overall, Pyjion enhances the execution speed of CPython by leveraging various optimizations in a JIT compilation context.

Pyston [48] is an alternative Python interpreter that includes a JIT step as well as many performance optimizations for the execution of Python programs (discussed specifically in Section 6.3). Pyston-lite is a derivative of Pyston that just retains the JIT step but with a focus on easier installation and setup. Pyston aims to achieve the highest performance possible; however, Pyston-lite may not match the same level of performance as the full implementation of Pyston. Yet, it still offers improved speed compared to using CPython as the interpreter: the authors claim 10% acceleration on macrobenchmarks [48]. It is important to note that Pyston is available for Python version 3.8.12, whereas Pyston-lite supports a wider range of Python versions: 3.7 through 3.10.

Let us note that in general, the performance of most tools in this section has been estimated in contrast to vanilla Python. Although this suits the use case scenario motivating the section, performance generally does not increase monotonically by combining the usage of multiple tools. For example, although Pyston-lite significantly improves the performance of vanilla Python, we observed that using it in combination with Numba parallelization tended to degrade the performance of Numba obtained with CPython (e.g., the execution time of the matrix multiplication code has more than doubled in some of our experiments).

4 ACCELERATING NUMERICAL LIBRARIES USAGE

In this scenario, an algorithm would have already been implemented by the data scientist, but contrasting with the previous section, it would not rely only on vanilla Python and would also use Python numerical libraries. Indeed, it would have been recognized that the problem depends mostly on standard data structures such as float matrices, thus aiming at benefiting from associated out-of-the-box primitives (e.g., matrix decomposition algorithms). In this section, we focus on means to provide faster execution of such numerical libraries or APIs.

For the sake of clarity and legibility, we focus on the three main libraries used in DS to facilitate and accelerate the development of single-threaded numerical computation code: NumPy [34], Pandas [56], and Scikit-learn [60]. These libraries are always among top results when looking for cornerstone libraries to carry out statistical and numerical computations needed for the practice of DS and ML. They were ranked 20th, 27th, an 103rd in terms of monthly downloads on PyPI in March 2023, respectively. It is worth mentioning that other libraries are widely used in DS and ML. However, they are tied to secondary tasks such as pre-processing (e.g., NLTK, ranked 334th) or visualization and plotting (e.g., Matplotlib, ranked 114th). As this survey focuses on accelerating DS code, we do not directly cover these libraries in this section. Statsmodels is also a frequently mentioned library, but we did not retain it, as it is a bit further down in the PyPI download ranking (327th).

Besides approaches covered in Section 3 (e.g., compilation, transformation), in the context of these libraries we mainly found solutions implementing an API with the same signature (same inputs and same outputs) as the original but proposing better performance. We refer to these as *drop-in* libraries. The execution of those drop-ins can be done using multiple CPUs, GPUs, and/or with a more efficient implementation. It may eventually require minor modifications such as data copies and changing function parameters. Ideally, they bear minimal cost to the practitioner in

terms of development overhead. In this section, we will review the three identified libraries and their performance enhanced counterparts.

4.1 NumPy

NumPy is one of the most used Python libraries, as it provides a multi-dimensional array format central to many other libraries. It also includes a set of routines for manipulating arrays with different operations, such as mathematical primitives, shape manipulation, and sorting. NumPy exploits BLAS and LAPACK and is therefore much faster than vanilla Python code. However, it under-utilizes parallel computer architectures. Several examples of NumPy drop-in libraries attempt to circumvent this issue.

4.1.1 Legacy Drop-in. Distarray [38] is a drop-in library for NumPy that distributes the execution of NumPy operations across multi-core CPUs, clusters, or supercomputers. It depends on IPython.parallel [80] (surveyed in Section 5.2) and MPI for setting up a cluster. Closely related is DistNumPy [43], which implements parallel NumPy operations by also using MPI underneath. DistNumPy was deprecated and moved to Bohrium, which is in active development.

Bohrium [42] is a runtime that maps NumPy array operations (universal functions, also known as *ufuncs*) onto different hardware platforms such as multi-core CPUs, GPUs, and clusters. To use Bohrium, the user must either replace the NumPy library import with the Bohrium library or launch a script with the command `python -m bohrium myscript.py`.[2] Bohrium uses different techniques to speed up computations. For example, Bohrium supports lazy evaluation, which means that NumPy operations are regrouped for evaluation until a non-NumPy operation is found. Bohrium fully supports NumPy views, and therefore no data copies are done when slicing arrays. When certain conditions are met, array operations are fused into a single kernel that is compiled and executed. Data copies between main memory and GPU memory are done only when the data is accessed through Python or a Python C extension.

Bohrium is built with components that communicate by exchanging a *vector bytecode* (an intermediate representation corresponding to the NumPy array operations). The instruction (original code) is passed to a *Bridge* component that generates the vector bytecode. This bytecode is passed to a *Vector Engine Manager* component that manages the data location, ownership of arrays, and the distribution of jobs between vector engines. The *Vector Engine* component is an architecture-specific implementation to execute the bytecode. Non-NumPy or unsupported operations fall back into the regular CPython interpreter. Althoughy setting up Bohrium is fairly straightforward, experimentally we found that its value as a drop-in for NumPy remains limited. For example, singular value decomposition, which is commonly used in the practice of DS, even causes a crash instead of gracefully falling back to NumPy.[3] In addition, we did not find cases where using the GPU as a backend did not lead to strong performance degradation.

D2O [78] is a middleware between NumPy arrays and a distribution logic. In that sense, it is not a drop-in library but an interface to provide parallel execution of NumPy array operations through the use of a *distributed data object* format. The user can pass a NumPy array as an argument to create a distributed data object, along with options regarding distribution strategy. The distributed data object supports many NumPy instructions, such as arithmetic operations, indexing, and slicing. D2O relies on MPI4Py to distribute the work (see Section 3). Therefore, to exploit parallelism with D2O, the user must create an MPI job. The number of nodes can be specified on the command to run the Python program. For lower-level instructions, the MPI library is accessible for code refactoring.

[2]Bohrium is incompatible with recent NumPy versions. Out tests were carried out with NumPy 1.21.5 (12/2021).
[3]Tested with the latest version on PyPI in March 2023.

4.1.2 GPU Acceleration. Many NumPy functions boil down to the same operation applied to large vectors or matrices, and can thus exploit GPU acceleration. CuPy [54] was designed to cover the API of NumPy as widely and transparently as possible. CuPy uses the NVIDIA CUDA framework and other CUDA libraries for optimization, such as cuBLAS, cuDNN, and cuSPARSE. Given the differences of memory management between the main memory and GPU memory, for harnessing the library at its best, the user must manually indicate data copies so that data is available in the GPU memory when CuPy functions are called. However, the process remains straightforward compared to CUDA programming. For example, using CuPy in the context of a semi-supervised learning task allowed to divide the baseline computation time by 6 with reasonable implementation efforts [17]. For cases where the available functions are not enough, CuPy supports creating user-defined CUDA kernels for two types of operations. One is for element-wise operations where the same operation is applied to all the data. The other operation is for reduction kernels, which folds all elements by a binary operator.

In the line of NumPy drop-in libraries for GPUs, there is also PyPacho [5] and DelayRepay [51]. PyPacho is based on PyCUDA and PyOpenCL. Although it is a promising tool, it is not as mature as CuPy and offers less compatibility. However, DelayRepay is a drop-in library and applies code optimization to accelerate its execution. DelayRepay has a delayed execution of NumPy operations because it analyzes them and tries to fuse them before execution. When a NumPy operation is found, it checks if its output is the input of another NumPy operation. If the rule is fulfilled, the operations are fused and the AST is modified. The NumPy operations are fused until a non-NumPy operation is found. When a non-NumPy operation is found, the fused AST node is compiled into a GPU kernel and executed in the GPU. This is a main difference compared to CuPy, which executes each operation individually.

Although not a drop-in library for NumPy, PyViennaCL [73] provides a set of equivalent operations to be executed in multi-core CPUs and GPUs. PyViennaCL is a wrapper for ViennaCL (written in C++), which is a linear algebra library and numerical computation to execute on heterogeneous devices. To use PyViennaCL, the user must import the library and use the constructs provided by the library. Similarly to DelayRepay, PyViennaCL uses delayed execution. Arithmetic operations are represented by a binary tree and are computed only when the result of the computation is necessary.

4.1.3 Compilation Based. The JAX [13] library provides composable transformations of Python programs based on NumPy. All JAX operations are implemented using the Accelerated Linear Algebra compiler (XLA) [74]. JAX provides a set of equivalent functions to NumPy. Therefore, it can be used as a drop-in library for NumPy. Besides performance improvement based on vectorization and parallelization, JAX provides JIT compilation into a GPU or tensor processing unit using the *jit* function. Another functionality is the evaluation of numerical expressions and generating derivatives (e.g., automatic differentiation by passing functions to the function *grad*), which is commonly used by gradient methods for training neural networks. Another important functionality in JAX is *vmap*, which is a mapping function to vectorize operations. The *jit* function can be applied to *grad* and *vmap* to obtain better performance results.

An option specialized in speeding up numerical expressions written in NumPy is NumExpr [23]. This library is compatible with a subset of NumPy operations. To use it, expressions are passed as a string to the library function *evaluate*. The expression is compiled into an object that contains the representation of the expression and the types of the arrays. To validate the expression, first it is compiled by the Python *compile* function, the expression is evaluated, and the parse tree is built. The parse tree is compiled into bytecode, and a virtual machine uses *vector registers*, each with the same fixed size. Arrays are handled as chunks, and these chunks are distributed among

the CPUs to parallelize NumPy operations. This approach has a better usage of cache memory and can reduce memory access, especially with large arrays.

In this inventory, we may also mention work surveyed in Section 3.3 like ALPyNA, Pythran, and Numba. These tools have general applicability for Python performance improvement but also provide performance improvements specific to NumPy.

4.2 Pandas

Pandas is a highly popular Python library for data analysis and manipulation. Its data frame format is widely used in DS, as it notably allows to handle heterogeneous data, time series, and query-based manipulation, to name a few features. A data frame is a two-dimensional data structure that contains labeled axes: row and columns. It is the primary data structure used in data analysis tools. Nonetheless, Pandas operations usually only use one core at a time when doing computations. Thus, multi-core CPU and GPU oriented drop-in libraries have emerged to accelerate Pandas-like operations.

Vaex is a library that contains a set of packages meant to optimize memory usage when managing large datasets [14]. Vaex-core is a drop-in library for Pandas-like operations on data frames. Similarly to Bohrium [42] with NumPy, most operations on Vaex are lazily evaluated, and they are computed only when needed. This reduces the amount of memory required compared to other similar libraries. Vaex also works with small chunks on data on the RAM, and therefore it can work with datasets larger than the typical RAM of a computer. It works best with files in HDF5, Apache Arrow, and Apache Parquet formats.

A multi-core CPU drop-in implementation of Pandas is Modin [61]. Modin can perform in a single node locally (multi-core CPUs) or in a cluster environment. Modin is based on a custom version of the Pandas data frame and treats operations on the data frames as user queries which are compiled. Having a similar design as relational databases which work with relational algebra, Modin is designed to work with a custom data frame algebra, aiming at simplifying and optimizing operations on a data frame. The Pandas-like API instructions are translated into data frame algebra with optimizations if possible. Then, the optimized query is passed to a subsystem called *Modin Dataframe*, which works as a middle layer between the query compiler and the actual execution backend. A data frame can be partitioned by columns, rows, or blocks depending on the operation required and the size of the data, enabling work distribution. In local mode, the number of partitions is by default equal to the number of available CPU cores. The Modin Dataframe subsystem passes the data to the execution layer where different execution engines can be used, such as Dask [71] or Ray [49] (see Section 5.2 for an introduction of the latter), which are in charge of the actual execution of computations on partitioned data in a task-based approach. The installation with the Ray backend is straightforward, with 40 times acceleration for a column concatenation benchmark but mitigated results otherwise. In addition, moderate code adaptation will generally be needed, as Modin return formats often slightly differ from their Pandas counterparts.

cuDF [68] is a Pandas drop-in library that runs on the GPU. It is used for manipulating data with the GPU for DS pipelines. cuDF is a building block of RAPIDS, a platform to execute ML and DS tasks in GPUs (see Section 4.3). Data frames can be created, read from files, and converted from Pandas data frames and CuPy arrays. Some tools, although not drop-in libraries for Pandas as such, bear high similarity to Pandas to such an extent that minor refactoring to the code can be used for the same purpose. Following this approach, we found Datatable [27] and Polars [62]. Datatable is implemented in C++ and uses multi-threading for certain operations to speed up processes. Polars lazily evaluates queries to generate a query plan and optimizes it so it can run faster and reduce the memory usage, possibly exploiting parallelism. Both libraries can also easily export to and from NumPy and Pandas formats.

4.3 Scikit-learn

Scipy [85] reuses the array format defined by NumPy but aims at a more comprehensive coverage of general-purpose mathematical and statistical concepts, such as linear algebra, statistical tests, and signal and image processing. Scikit-learn builds upon NumPy and Scipy by implementing many models from the ML literature, such as regression, classification, and clustering models. Most models implement *fit* and *predict* functions, providing a unified API for the library.

4.3.1 dislib. Dislib [87] is an ML library for Python to be executed in high-performance computing clusters. Dislib is built on top of PyCOMPSs [82] (a task-based parallelization library presented in Section 5.2 in the context of the first scenario) and exposes two main components to the developers: (1) an interface for distributed data handling and (2) an estimator-based API. The data-handling interface provides an abstraction to handle data as a dataset that can be divided in multiple subsets to be distributed and handled in parallel. Datasets can be given as NumPy arrays for dense data and Scipy compressed sparse row matrices for sparse arrays. Its wrapping Dataset format is the input for the ML models.

The estimator API provides a set of ML models with a similar syntax as Scikit-learn. An estimator is an abstraction of an ML model and typically implements two characteristic methods in Scikit-learn: fit and predict. To summarize, data is loaded into the Dataset format. An instance of an estimator object (representing the ML model) is created, and the fit function is invoked with its parameters. The estimator object is used to retrieve information of the trained model and generate predictions. Dislib authors report that they outperform MLlib [47] for a k-means clustering task as the sample size, and the number of computation nodes used grows very large.

4.3.2 cuML. RAPIDS [55] is a set of libraries for data manipulation and ML developed on top of the CUDA language and thus is aimed at the execution of DS pipelines in GPUs. In this set of libraries, cuML is strongly related to Scikit-learn. As CuPy aims at covering most of the NumPy API, cuML was created with the target to cover as much of the Scikit-learn API as transparently as possible. Similarly, as Scikit-learn is built on top of the NumPy and Pandas formats, cuML exploits the CuPy array and cuDF data frame formats, respectively. Most of its API can also be executed in a distributed environment using Dask, a task-based framework relevant to the scenario presented in Section 5. The cuML developers report that GPU implementations can run up to 50 times faster than their Scikit-learn CPU-based counterpart.[4]

4.3.3 MLlib. MLlib [47] is an ML library part of the Spark system. It is similar to Scikit-learn with a set of ML models and data processing instructions. Built on top of Spark, it thus comes with the Spark installation and a Python API to use it. The implementation of algorithms is parallelized so that large data processing jobs exploit data distributed on Hadoop clusters.

5 STRUCTURING FRAMEWORKS

In this section, we consider high-performance libraries and frameworks which impose a specific way of thinking and programming to the practitioner and are thus preferably used right when implementation starts.

5.1 Deep Learning Frameworks

Many models used in DS can be formalized as **Directed Acyclic Graphs (DAG)** (e.g., Bayesian networks, probabilistic mixture models, and, most notably, neural networks). A range of Python libraries, commonly referred to as *deep learning frameworks*, come with specialized support and

[4] https://github.com/rapidsai/cuml

useful abstractions to practitioners needing to put this kind of models in action. Computations underlying DAGs are typically embarrassingly parallel: benefiting from high-performance computation devices such as multi-core CPUs or GPUs is therefore an implicit requirement of these libraries. Technically, they are symbolic mathematical libraries which allow to define arbitrary computational DAGs along which data is transformed. However, their *deep learning* label is often well deserved, as they provide many facilities specifically oriented toward neural networks, such as automatic gradients and back-propagation at DAG nodes, enabling fitting model parameters to input data. At runtime, the computational graphs and all functions which operate on them (e.g., custom loss functions and gradient optimizers) are compiled and loaded to the GPU. The training procedure then triggers kernel execution on the GPU. These frameworks can also be run on multiple cores of a CPU when no compatible GPU is available; however, taking effective advantage of the GPU generally yields very significant speedup. For example, the computation time of a piece of TensorFlow code heavily relying on the CPU can be divided by 10 if correctly using the GPU API [17].

TensorFlow [1] is the most prominent in this range of tools. Besides offering a wide range of ready-to-use model architectures (sometimes even along pre-trained model weights), TensorFlow defines a comprehensive API to program custom components then compiled and loaded on the GPU, such as model structures, loss functions, or optimizers. As this code is meant to be loaded on the GPU, although it uses the Python syntax, it cannot be mixed with regular Python instructions, which causes additional implementation effort. In TensorFlow, the computational DAG is defined statically so that its compilation and execution yields maximum performance at runtime. The explicit definition of the computational graph and its asynchronous execution on the GPU yields constructs which tend to diverge from Python standards. Mastering TensorFlow therefore takes some time and practice.

Torch is another deep learning framework, developed by Meta with the similar aim to support neural network model training. However, it is based on the Lua language, which is limiting its popularity. PyTorch [58] is the port of Torch to Python, motivated by the will to keep its API and basic principles. PyTorch came to the market after TensorFlow but has gained momentum and is catching up in terms of popularity (8M monthly downloads vs 15M for TensorFlow according to PyPI statistics).[5] Good documentation facilitates its adoption by newcomers, and it offers many ready-to-use model architectures and pre-trained parameters. PyTorch has built-in high-level APIs, which are delegated to Keras in the case of TensorFlow. Pure TensorFlow requires significant non-standard boilerplate code development in comparison.

In TensorFlow, the computational graph is defined and compiled statically, and placeholder data is replaced at runtime. PyTorch offers more control at runtime, such as allowing to modify execution nodes at runtime in ways forbidden by TensorFlow, facilitating the implementation of sophisticated training loops. Language constructs are closer to Python standards, with object-oriented constructs meant to be familiar to experienced programmers. Overall, its APIs are less rigid, but this comes at the cost of more code to write and generally slightly longer execution time for equivalent tasks.

This distinction between static and dynamic computational graphs has other consequences, first in the way TensorFlow and PyTorch handle variable-sized input data. Due to the static computation graph approach, doing so is difficult with TensorFlow. The TensorFlow Fold tier library offered limited support, but it is no longer maintained. In contrast, this is built-in in PyTorch.

Debugging PyTorch is also straightforward, although it is more difficult with TensorFlow due to the static graph definition. In the latter case, this requires mastering a specific debugging tool,

[5] https://pypistats.org was accessed on October 28, 2022.

tfdbg. To compensate, TensorFlow comes with TensorBoard, which packages visualization and monitoring tools. In PyTorch, to come up with equivalent features, custom graphs have to be built using, for example, Matplotlib or an interactive plotting library like Dash. More facilities exist for distributed training in TensorFlow, as well as deployment to production servers, and embedding in limited-resource devices such as mobile and Raspberry Pi using TensorFlow Lite. Finally, TensorFlow supports several languages beyond Python (including C++ and Java), whereas PyTorch focuses on Python.

Theano [4] offers very similar features to TensorFlow and PyTorch, primarily aimed at defining and training neural network structures. It has been around since 2007, but its development has been stopped—the latest release dates back to 2020. It has been forked and repurposed to Aesara [64], the latter being aimed at optimizing and evaluating mathematical expressions involving numerical arrays and symbolic inputs. Aesara has therefore more general applicability, comparable to numerical libraries such as NumPy (see Section 4.1), but it involves computational graphs and therefore cannot be included in regular Python projects in a straightforward way.

Frameworks such as TensorFlow and Theano require asynchronous thinking and significant overheads in boilerplate code development. Keras [19] came as a high-level library meant to accelerate experimentation with such deep learning frameworks. It abstracts them with convenient input-output primitives and a simpler training API, notably while allowing to access and parameterize the backend framework. Overall, it allows data scientists to program in a more procedural fashion. Up until version 2.3,[6] it supported multiple frameworks: TensorFlow, Theano, and the Microsoft Cognitive Toolkit (formerly known as CNTK). Since version 2.4, only TensorFlow is supported, which means it is now part of the TensorFlow distribution, in practice.

MXNet [18] claims high flexibility and scalability, notably supported and used internally by Amazon. Like TensorFlow, MXNet supports several languages beyond Python (C++, Python, R, Scala, Matlab), whereas PyTorch focuses on Python. It offers a flexible frontend, with an imperative API meant to be familiar to newcomers and a symbolic API aimed at maximizing performance. However, it lacks high-level IO primitives compared to PyTorch and Keras, which is detrimental to quick adoption.

5.2 Distributed Computation Frameworks

An approach used by multiple Python-intensive computation libraries is task-based parallelization, especially when large sets of data are involved. The task-based approach refers to a strategy where the work is divided into multiple tasks, and these tasks are handled by a task manager that assigns them to threads that execute them. The execution of a program is a sequence of tasks, and in some cases independent tasks can be executed in parallel. Usually, the task-based approach is implemented with a queue of tasks, a thread pool where threads wait for a task assignment, and some message protocol (i.e., MPI) to communicate data and instructions between tasks and the task manager. Although of general applicability, most libraries in this section impose in-depth modifications to an existing codebase and require heavy software setup. This contrasts with task-based approaches reported in Section 3.2, where the most complex elements are scaffolded by using decorators. This makes them a more suitable choice if algorithm implementation has not started yet.

Directly relating to deep learning frameworks presented in the previous section, Horovod [75] aims at facilitating the usage of distributed resources (i.e., multiple computation nodes, potentially each holding multiple GPUs) by these frameworks. Indeed, deep learning frameworks are sometimes packaged with modules dedicated to distributed training, but, in the case of TensorFlow for example, they are rigid and difficult to set up. Horovod compensates this problem while offering

[6]Released October 7, 2019.

the support to multiple frameworks (including TensorFlow, Keras, PyTorch, and MXNet). Behind the scenes, Horovod relies on a message passing layer, which, for example, can be OpenMPI (presented in Section 3). The default is to use Gloo [37], a communication library developed by Meta. The authors claim up to 90% scaling efficiency, depending on the neural architecture at hand.[7]

Some task-based parallel Python libraries we found are wrappers of an already existing library in a different language. This is the case of torcpy [32] and Charm4py [30]. Both libraries are wrappers of their C/C++ counterpart library; torcpy for TORC [33] and Charm4py for Charm++ [41]. In both libraries, the parallelism is expressed by using the library instructions and an API lets the programmer orchestrate asynchronous tasks and distributed objects. In torcpy, tasks are executed by launching multiple MPI processes using one or multiple worker threads. It builds upon MPI4Py (see Section 3.1) and implements an API meant to upscale the logic underlying *multi-processing* or *concurrent.futures* packages to benefit from high-performance clusters. Multi-processing or concurrent.futures packages and Python built-in packages aim at overcoming limitations imposed by the GIL, by implementing a parallel *map* function and allowing the asynchronous execution of functions in multiple parallel Python processes, respectively. Torcpy claims 90% computation efficiency on a Monte Carlo molecular simulation tasks with 1,024 compute nodes. Torcpy is based on Python dictionaries for its data management; however, in Charm4py, multiple distributed objects are executed and coordinated in a unit called the *processing element*. Distributed Python objects are implemented and allow remote method invocation using message passing. To overpass the GIL of only one thread, the implementation of Charm4py launches the Python executable in multiple nodes or even multiple times on the same node thanks to this distributed object mechanism. Unfortunately, it does not seem to run anymore in recent Python environments.[8]

There are also task-based parallel libraries written mostly or entirely in Python, such as SCOOP (Scalable Concurrent Operations in Python) [36], Parallel Python [57], Celery [21], and Playdoh [72]. SCOOP is similar in spirit to MapReduce frameworks [28], as its API revolves around *map* and *reduce* functions. Asynchronicity and parallelization are enabled by a custom implementation of the built-in Python *futures* class. Specifically, their workers act as independent elements that interact with a broker to mediate their communications. The documentation gives extensive instructions to facilitate the deployment on high-performance clusters. Parallel Python also relies on its own library constructs to express parallelism by submitting job passing functions and general execution information as parameters. The library is meant to overcome limitations imposed by the GIL when using Python's built-in threading library and exploit computation clusters. The authors claim automatic discovery of computational resources and their dynamic allocation. However, the library is available neither on PyPI nor on GitHub but only on a website [57]. Celery is a distributed task queue system that can be used to complete heavy DS and ML computations, but it is also meant to have wider applicability in view to support large business applications. Its main components are its broker and backend. The broker is responsible for managing communication between computation threads, and the backend provides the memory storage for queue management. The cost of this flexibility is that deploying Celery is much more complex that alternative approaches mentioned in this section, and thus is preferred for complex business logic but probably not for prototypical DS and ML projects. It is actively maintained and backed by a large community. The main feature of Playdoh [72] is a parallel and distributed map function, as most libraries in this section. It is mainly oriented toward numerical optimization and Monte Carlo simulation, with some specialized functions in this area. In addition, if the tasks are made of PyCUDA or CUDA code, Playdoh can distribute the work to several GPUs in parallel. It is worth

[7] https://horovod.readthedocs.io/en/stable/summary_include.html
[8] Tested on March 30, 2023 with Python 3.9.

noting that Parallel Python and Playdoh are not actively maintained, and not supported by Python 3+ interpreters. Formerly known as IPython.parallel, Ipyparallel [80] is a Python library for the development of task-based parallel applications. This package leverages the usage of IPython engines in parallel to run tasks. It has four main components: engine, hub, schedulers, and client. The engine is a subclass of the IPython kernel for Jupyter and is responsible for running user tasks as commanded by a scheduler. The hub manages the cluster by keeping track of schedulers and clients. With this architecture, Ipyparallel allows to abstract potentially heterogeneous distributed computation facilities, accessed by multiple clients working collaboratively. However, its powerful abstractions require significant boilerplate code, preventing straightforward adaptation of existing DS and ML projects.

Asynchronous function execution is central in Parsl [6]. Its task-based distributed programming model is based on Parsl *apps*, which may be decorated Python functions (*@python_app*) or calls to shell commands (*@bash_app*). Like for torcpy [32] and SCOOP [36], task distribution is enabled by a custom *futures* implementation to manage asynchronous function execution in a distributed context. Its specificity is to facilitate function chaining, which enables parallelization of complex jobs. An Executor is deployed on each host of a distributed architecture, each managing several local workers. Available resources and task distribution are abstracted by a dataflow kernel on the client side. A set of launchers accommodate for various high-performance cluster types. Parsl is backed by an active community.

Ray [50] is a versatile task-based parallelization tool. On the one hand, it provides low-level facilities, based on primitives like actors and tasks that allow to define and manage distributed computations. Tasks are stateless and executed asynchronously, whereas actors represent stateful computations. On the other hand, it also provides high-level libraries for deep and reinforcement learning, data processing, and analytics. Ray is widely adopted, serving as a parallel framework for other libraries like Modin, LightGBM, and Mars. Although it provides direct support for frameworks such as TensorFlow and PyTorch, it can also act as a communication layer for Horovod. Ray is straightforward to install and get up and running in its simplest configuration (multi-core CPU), but it requires investment for complex configurations with multiple computation nodes, each possibly holding multiple GPUs. It is therefore rather meant for practitioners with production needs.

Dace [10] is a Python library that translates Python code to C++ using the *@dace* decorator. It transforms the code into a **Stateful DataFlow multiGraph (SDFG)**, supporting a subset of Python code, NumPy operators, and explicit dataflows. The SDFG is a directed graph where nodes represent containers or computations, and edges indicate data movement. Dace has two types of containers: data (memory-mapped arrays) and stream (concurrent queues). Computation containers contain stateless functions. The Python to C++ compiler in Dace leverages the Python AST to infer types and shapes, and perform code analysis. SDFGs enable parallelism by grouping subgraphs, and optimizations are applied through graph transformations. Compilation involves inferring data dependencies, hierarchical code generation, and invoking the compiler for the desired output. Although basic usage is simple on multi-core CPUs, complex scenarios require additional development. Dace heavily relies on the host software environment and may be harmed by version clashes. GPU and field-programmable gate array support is possible but requires advanced knowledge of the library.

Dask [26, 71] is a widely used task-based distributed computing library closely integrated with NumPy and Pandas. Although it shares a similar API with these libraries and could arguably be considered as a drop-in library (see Section 4), it introduces task-based logic and incurs setup overheads. Dask provides APIs for arrays (similar to NumPy), data frames (similar to Pandas), and lists (similar to Python iterators). It utilizes task schedulers to split arrays or data frames into smaller pieces, distribute work, and merge results. Computation is represented as a DAG where tasks are

defined as function-argument tuples and can be executed concurrently. Dask supports various task schedulers for single or multiple nodes in a cluster, allowing customizable configuration. Although Dask does not directly support GPUs, it can schedule GPU-related work at the task level using Dask-cuDF, which extends the cuDF data frame library (see Section 4) within Dask.

The Tuplex library [77] exhibits a similar API to Dask and utilizes optimized LLVM bytecode generation to achieve maximum acceleration. Tuplex performs a dynamic analysis by considering both the code and the data for code generation. Impressive results have been reported, demonstrating up to 91 times acceleration in intricate data pre-processing tasks involving user-defined functions encompassing operations like regular expressions and query joins. The present work primarily focuses on DS and ML jobs with intense numerical computations; however, it is noteworthy that complex pre-processing and business tasks can assume critical importance in the management of large-scale data within production environments. Therefore, Tuplex remains a relevant tool, although its improvements may not be considered spectacular in the context of typical research ML projects.

6 DISCUSSION AND RESULTS

In this article, we have presented numerous tools and techniques that propose enhancement of the performance of Python in the context of DS and ML. We have tried to depict those tools from the perspective of practitioners to provide them with sufficient insights to select and use an appropriate tool in this still-ongoing quest for Python performance enhancement. We thus have infused the need of practitioners into stereotypical scenarios and assigned existing tools and approaches to the most relevant scenario at hand.

6.1 Results

We have identified different kind of techniques during our survey that we shortly summarize here:

- Parallelization libraries—MPI, OpenMP, and task based
- Drop-in libraries
- Program transformation—transpilers, JIT, general compilers (e.g., LLVM based)
- Complete frameworks.

6.1.1 First Scenario: Pure Python Performance Improvement. The tools relevant to this user scenario are summarized in Table 2. In this table, the surveyed tools are characterized by the following:

(1) Tool name and reference.
(2) The implementation technique for performance enhancement (based on the aforementioned list of techniques).
(3) Supported acceleration on a CPU, a GPU or both.
(4) Usage complexity, which is a qualitative judgment of the effort to put in the effective usage of the tool. It combines an evaluation of the time needed to master its API, its impact on the structure of an existing piece of code, and the efforts needed for its deployment. The number of plus (+) signs denotes the complexity, getting 3+ means that the tool is complex to learn and potentially intrusive in code and may require a lot of tweaks. Getting a minus (–) sign means that the tool requires little work beyond few command lines or editing a configuration file, for example.
(5) Any additional limitation or requirement.

The first scenario assumes the existence of a pure Python codebase, which must be accelerated and parallelized. Therefore, it is mainly relying on parallelization libraries and program

Table 2. Summary Information of Tools from the First Scenario

Tool Name	Activity Period	Technique	Hardware	Usage Complexity	Open Source	Maintained by	Popularity
MPI4Py [25]	10.2006–11.2022 ★	MPI	CPU	+++	Y	Individuals	632/174,734
Comments	Used by other libraries and software such as H5py, vtk, dask, and PocketFlow. DS/ML practitioners may prefer using a library that hides low-level MPI instructions to obtain parallelism.						
PyPar [70]	06.2007–11.2016 ∗	MPI	CPU	+++	Y	Individuals	68/na
Comments	Used by other projects such as ANUGA, TCRM, and Wind multipliers. Helps to avoid the construction of memory buffers by using Pickle. Not compatible with Python 3+.						
Pymp [45]	05.2016–03.2022 ★	OpenMP	CPU	++	Y	Individuals	256 /66,880
Comments	Only work on systems with fork support. Overhead to circumvent the GIL by using the OS fork method.						
PyOMP [46]	06.2021–01.2023 ∗	OpenMP	CPU	++	Y	Individuals	33/na
Comments	Only works with Linux x86-64 with one specific Python and NumPy version. It is distributed as a forked Numba package with a version out of the main branch.						
PyCOMPSs [82]	01.2017–11.2022 ★	Task based	CPU	+	Y	University	na/96
Comments	Wrapper for COMPS [81]. Used mostly in academic environments. The user needs to know task-based programming and use the definitions of the library.						
Pygion [76]	na	Task based	CPU	+	na	na	na
Comments	Wrapper for Legion [9].						
PyKokkos [3]	04.2021– 08.2022 ∗	Task based	CPU	+	Y	Individuals	58/41
Comments	Wrapper for Kokkos [84]. Works only with Ubuntu with gcc 7.5.0 and NVCC 10.2.						
Jug [20]	07.2019–05.2022 ★	Task based	CPU	++	Y	Individuals	390/567
Comments	The user requires familiarity with task-based programming and library definitions.						
Pydron [52]	01.2016–01.2016 ★	Task based	CPU	++	Y	Individuals	5/1
Comments	The user requires familiarity with task-based programming and library definitions.						
Cython	08.2007–01.2023 ★	Compiler	CPU	+++	Y	Individuals	7.9k/30,221,789
Comments	To easily create a C extension for Python, including parallel processing. Used by many popular libraries such scipy, pandas, and NumPy. Requires manual refactoring of code for performance improvement.						
Numba [44]	12.2017–12.2022 ★	JIT	Both	+	Y	Enterprise	8.5k/2,321,402
Comments	According to the statistics it is a highly popular package. It uses decorators to give hints to the compiler and has a dependency on LLVM.						
Hope [2]	10.2014–05.2018 ★	Transpilation	CPU	+/−	Y	University	384/167
Comments	Used mainly in academic environment as support for other packages such as PyCosmo. Only supports a subset of Python.						
Shed skin [22]	07.2009–03.2023 ∗	Transpilation	CPU	++	Y	Individuals	673/35
Comments	All variables are implicitly typed. Only supports Python 2.4–2.6.						
Nuitka [35]	02.2013–01.2023 ★	Transpilation	CPU	–	Y	Individuals	8.7k / 66,296
Comments	Translates Python code to highly optimized C or C++ code. Used by at least 25 other packages.						
Pythran [31]	08.2012–01.2023 ★	Compilation	CPU	–	Y	Individuals	1.9k/262,749
Comments	Used by at least 13 packages on GitHub to accelerate computations. Generates C++ code and optimizes it. The code needs to be compiled to be used in another project as a module.						
Transpyle	12.2017–08.2019 ★	Transpilation	CPU	+ (annotations)	Y	Individuals	122/78
Comments	Can bridge Python, Fortran and C/C++. Only a subset of the language is supported.						
Autoparallel [67]	11.2017–08.2020 ∗	Compilation	CPU	+	Y	Individuals	11/na
Comments	Relies on PyCOMPSs and PLUTO. The code is annotated and then automatically translated into PyCOMPSs task definition.						
Pyjion [63]	11.2020–11.2022 ★	JIT	CPU	+++	Y	Individuals	1.3k/894
Comments	Compiles CPython bytecode into improved machine code. Does not currently support with blocks and async...await (YIELD_FROM) statements.						
Pyston-lite [48]	03.2021–03.2023 ∗	JIT	CPU	–	Y	Enterprise	2.4k/272,262
Comments	JIT extension to CPython. Usage is straightforward but not frequently updated on PyPI. Needs to be built for a recent version, soon to be incorporated to CPython.						

Activity period format: month.year. ★ First and last release on PyPI · ∗ First and last commit on Github · ± First and last update on SourceForge.
Popularity: GitHub stars/PyPI last month downloads. Statistics as of March 2023.

transformations. However, due to their genericity, some of the tools described in this scenario could also apply to other scenarios. Notably, some tools are already applicable for the enhancement of performance of specialized DS libraries (e.g., ALPyNA, Numba). As we can see in Table 2, some of the tools rely on task-based parallelization behind the scenes. Using the latter as a structuring framework generally comes with technical complications (see Section 5.2), but the tools surveyed in Section 3 scaffold this complexity as much as possible. Alternatively, some of the proposed tools act as wrappers from existing C/C++ libraries already offering great performance.

Transpilation and compilation approaches offer to hide some of the complexity for the practitioner. The simpler ones do not require anything from the practitioner, except doing the compilation. The most advanced ones are relying on code annotation to guide the compilation to perform acceleration and parallelization. In general, all those approaches require more involvement of the practitioner to make them work, being potentially quite intrusive on the code through high refactoring (e.g., MPI-based techniques). Finally, very few propose to exploit a GPU, as it is known as a complex case for general-purpose programming.

6.1.2 Second Scenario: Accelerating Numerical Libraries Usage. The tools relevant to this user scenario are summarized in Table 3. In this context, it is assumed that the existing codebase relies on one of the most commonly used computation libraries: NumPy, Pandas, or Scikit-learn. Tools and approaches presented in Section 4 aim at enhancing or replacing these libraries.

In Table 3, we can see that most of the found approaches are drop-in libraries that replace as much as possible the syntax of the original library, keeping the same semantic but providing enhancement. Their usage is sometimes as simple as function call substitution. A few tools provide the exploitation of GPU devices for performance acceleration. For maximal benefits, they require additional operations relating to memory movement between central and GPU memory. In the context of CuPy, it materializes as copying NumPy arrays in CuPy ones. For example, Scikit-learn relies on NumPy and Pandas, whereas cuML relies on CuPy and cuDF to offer a broad coverage of the former. Many drop-in alternatives exist for NumPy, which is explained by the very high popularity of NumPy as a building block for DS and ML code development, and as a dependency in other Python libraries.

6.1.3 Third Scenario: Structuring Frameworks. The tools relevant to this user scenario are summarized in Table 4. Section 5 surveys tools which deeply affect an existing codebase and thus should preferably be used right when the implementation of a DS or ML algorithm starts. As a counterpart, they generally provide many primitives which facilitate the work of the practitioner if they stick to the framework driving principles. We framed deep learning frameworks in this category, as they come with their very own logic to which the data scientist must adapt. In exchange for this effort, they come with high-level abstractions and scaffold the access to GPU hardware so that maximal performance is obtained with minimal specific development effort.

In Section 5, we also gather distributed computing frameworks. They generally have wider applicability compared to deep learning frameworks and sometimes act as a backend for tools summarized in Section 6.1.1. However, when used in the first intention, they come with specific code constructs which heavily constrain software development, as well as complex setup procedures to deal with variable cluster configurations. As a consequence, it is generally better to involve these tools when implementation starts. Using these frameworks then pays off in terms of the size of the datasets they can handle, which can be orders of magnitude larger than with other tools surveyed elsewhere in this article.

Table 3. Summary Information of Tools from the Second Scenario

Tool Name	Activity Period	Library/Technique	Hardware	Usage Complexity	Open Source	Maintained by	Popularity
Distarray [38]	04.2014–10.2015 ★	NumPy/Drop-in	CPU	+	Y	Individuals	4/15
Comments	Distributed NumPy arrays based on MPI. No updates since 2015.						
DistNumPy [43]	04.2008–09.2011 ∗	NumPy/Drop-in	CPU	–	Y	Individuals	5/na
Comments	The project is deprecated, moved to Bohrium.						
Bohrium [42]	11.2017–11.2020 ★	NumPy/Drop-in	Both	+	Y	Individuals	218/336
Comments	Bohrium is used as support for other packages such as Veros and Weld. Drop-in replacement for NumPy but with limited coverage, claimed fall back to NumPy, observed crashes experimentally.						
D2O [78]	05.2016–09.2017 ∓	NumPy/Drop-in	CPU	+	Y	Individuals	na/na
Comments	Last update since 2017. Manual refactoring is needed to use the distributed data structure.						
CuPy [54]	06.2016–01.2023 ★	NumPy/Drop-in	GPU	+	Y	Enterprise	6.9k/22,849
Comments	Good NumPy coverage but not complete. Used by other libraries and part of the RAPIDS ecosystem. The user must add instructions for data copies between the GPU memory and main memory. CuPy has many packages for different CUDA and AMD ROCm versions.						
PyPacho [5]	04.2018–02.2021	NumPy/Drop-in	Both	++	Y	Individuals	1/na
Comments	Numerical computations based on OpenCL and CUDA. Not documented.						
DelayRepay [51]	xx.2020–xx.2020	NumPy/Drop-in	Both	–	na	na	na
Comments	Method remains conceptual, as no associated code can be found.						
PyViennaCL [73]	03.2014–05.2014 ★	NumPy/Drop-in	Both	–	Y	Individuals	32/27
Comments	Provides functions to create data structures (Vector and Matrix) and apply mathematical operations on multi-CPUs and GPUs.						
JAX [13]	12.2018–01.2023 ★	NumPy/Drop-in/JIT	Both	–/++	Y	Enterprise	23,000/4,135,872
Comments	Uses Accelerated Linear Algebra compiler (XLA) [74]. Used by many packages such as ColabDesign, trax, ml-workspace, and TensorNetwork. It can be used as drop-in library for NumPy. Nonetheless, JAX arrays are always immutable. Additionally, JAX is aimed to work best with functional programming.						
NumExpr [23]	01.2009–10.2022 ★	NumPy/Library	Both	+	Y	Individuals	2k/2,975,123
Comments	Used by other popular packages such as Pandas, zipline, and osmnx. The API is not as extensive as NumPy. Requires to work with large arrays to overcome the overhead of compilation and usage.						
Vaex [14]	02.2015–12.2022 ★	Pandas/Drop-in	CPU	–/+	Y	Individuals	7.9k/52,307
Comments	Used by at least 33 GitHub packages such as neural-lifetimes, geospatial-ml, radis, and optimus. This tool prioritizes memory optimization for handling large datasets, which may impact performance.						
Modin [61]	06.2018–01.2023 ★	Pandas/Drop-in	CPU	–	Y	Individuals	8.6k/1,390,926
Comments	Can use execution engines like Dask [71] or Ray [49]. Used by at least 47 GitHub packages such as aws-sdk-pandas, ludwig, and pandera. Modin lacks full implementation of certain Pandas functions, invoking unsupported functions incurs overhead as execution falls back to Pandas and results are transferred back to Modin, requiring data type transformation.						
cuDF [68]	11.2018–02.2023 ∓	Pandas/Drop-in	GPU	–/+	Y	Enterprise	5.5k/2,212
Comments	It is part of the rapids.ai collection of software. cuDF is restricted to use only with NVIDIA GPUs. It can lead to certain memory constraints. Specifically, due to the relatively smaller size of GPU memory, users may encounter overheads related to memory swaps or copies, which can impact performance.						
datatable [27]	03.2018–07.2021 ★	Pandas/Library	CPU	+	Y	Individuals	1.7k/73,586
Comments	Used by at least 3 other packages in PyPI.						
polars [62]	03.2021–03.2023 ★	Pandas/Library	CPU	+	Y	Enterprise	16.9k/466,445
Comments	Used by 186 packages and 1,664 repositories on GitHub. There are some differences between the Pandas API and the polars API.						
Dislib [87]	02.2019–11.2022 ★	SciKit/drop-in	CPU	++	Y	University	41/411
Comments	Provides a set of distributed algorithms (related to ML and data processing) such as regression techniques, k-nearest neighbors, and k-means. The library has been implemented on top of PyCOMPSs programming model.						
cuML [55]	01.2019–06.2020 ★	SciKit/Library	GPU	++	Y	Enterprise	220/2,059
Comments	Compatible with CuPy for NumPy support. Used by at least 6 packages and 178 repositories on GitHub. Does not convert code to run into GPUs, helps with deployment and management of Dask workers.						
MLlib [47]	10.2012–04.2023 ∓	SciKit/Library	CPU	+	Y	Enterprise	na / na
Comments	Must be used within Spark. Part of Spark, installed as pyspark. MLlib is part of spark, and in PyPI it is named pyspark.						

Activity period format: month.year. ★ First and last release on PyPI · ∗ First and last commit on GitHub · ± First and last update on SourceForge.
Popularity: GitHub stars/PyPI last month downloads. Statistics as of March 2023.

Table 4. Summary Information of Tools from the Third Scenario

Tool Name	Activity Period	Techniques	Hardware	Usage Complexity	Open Source	Maintained by	Popularity
TensorFlow [1]	12.2016–03.2023 ★	Framework, JIT	Both	+++	Y	Enterprise	175k/15,397,352
Comments	Used by 4,934 packages and 275,967 repositories on GitHub. Powerful and widely adopted, steep learning curve, industry standard.						
Keras [19]	11.2015–03.2023 ★	Library	Both	+	Y	Enterprise	58k/10,078,784
Comments	Formerly supported several frameworks, bound to TensorFlow since 2019.						
PyTorch [58]	12.2018–03.2023 ★	Framework, JIT	Both	++	Y	Enterprise	67.2k/9,992,796
Comments	Used by 225,826 repositories on GitHub. Major deep learning framework, high flexibility, more popular in research and academic contexts.						
Theano [4]	11.2010–07.2020 ★	Framework	Both	++	Y	University	9.7k/229,250
Comments	Used by 203 packages and 13,754 repositories on GitHub. Former competitor of TensorFlow, development stopped in 2020. Forked by Aesara [64] since.						
MXNet [18]	05.2017–03.2022 ★	Framework, JIT	Both	+++	Y	Foundation	20.4k/351,957
Comments	Used by 94 packages and 6,381 repositories on GitHub. Similar to PyTorch in terms of flexibility, lacks convenient IO primitives, and the release frequency has slowed down recently.						
TorcPy [32]	10.2019–12.2021 ∗	Task based	CPU	++	Y	Enterprise	16/na
Comments	Extending Python built-in CPU parallelization to cluster environments but has little adoption.						
Horovod [75]	08.2017–02.2023 ★	Task based, MPI	GPU	+	Y	Enterprise	13.3k/71,025
Comments	Used by 22 packages and 892 repositories on GitHub. Distributed deep learning framework based on OpenMPI or Gloo for communication.						
Charm4py [30]	11.2018–09.2019 ★	Task based	CPU	++	Y	University	280/162
Comments	Task-based distributed computing framework with distributed Python objects. Note that Python 3+ is not supported.						
SCOOP [36]	01.2013–08.2022 ★	Task based	CPU	++	Y	University	591/1,415
Comments	Used by 6 packages and 326 repositories on GitHub. Similar to a MapReduce framework, with extensive information available for deployment on high-performance clusters.						
Parallel Python [57]	na–na ∓	Task based	CPU	++	Y	na	na/na
Comments	Job-based distributed computing framework. It is absent from main repositories, and Python 3+ is not supported.						
Celery [21]	04.2009–02.2023 ★	Task based	CPU	+++	Y	Enterprise	21.5k/6,198,447
Comments	Used by 1,515 packages and 102,009 repositories on GitHub. Popular distributed task queue system with many features but can be complex to deploy.						
Playdoh [72]	03.2010–02.2011 ★	Task based	Both	++	Y	University	67/17
Comments	Distributed map function that also supports CUDA code. Not maintained since 2011 and does not support Python 3+						
Ipyparallel [80]	04.2015–03.2023 ★	Task based	CPU	++	Y	University	2.4k/143,330
Comments	Powerful but requires significant code adaptation.						
Parsl [6]	02.2017–03.2023 ★	Task based	CPU	++	Y	University	376/416,826
Comments	Task-based parallelization library that uses asynchronous function invocation, backed by an active community.						
Tuplex [77]	06.2021–10.2022 ★	Task based, Compil.	CPU	++	Y	University	798/90
Comments	Task-based and compilation-based framework for CPU parallelism. Requires the use of its own Domain Specific Language (DSL).						
Dask [26, 71]	01.2015–03.2023 ★	Task based	Both	+/++	Y	Enterprise	10.9k/7,016,339
Comments	Used by 1,583 packages and 47,013 repositories on GitHub. Supports both single-machine and distributed computing. Strongly bound to NumPy and Pandas.						
Ray [50]	06.2017–03.2023 ★	Task based, Backend	Both	+++	Y	Enterprise	25.8k/2,095,907
Comments	Used by 553 packages and 9,355 repositories on GitHub. Task-based framework with a versatile backend supporting both single-machine and distributed computing. Powerful but can be complex to set up effectively.						
Dace [10]	03.2019–03.2021 ★	Task based	Both	+++	Y	University	378/8,348
Comments	Used by 3 packages and 6 repositories on GitHub. Supports both single-machine and distributed computing. Powerful capabilities but requires significant work overhead for complex use cases and in a GPU context.						

Activity period format: month.year. ★ First and last release on PyPI · ∗ First and last commit on GitHub · ± First and last update on SourceForge.
Popularity: GitHub stars/PyPI last month downloads. Statistics as of March 2023.

6.2 Scope and Limitations of the Study

To mitigate the risk of being biased by our own research, we tried to be as open as possible following a simple narrative process. In addition, the narrative review allows us to provide DS and ML practitioners with an overall view on the different existing techniques. It is also sufficiently open to interest practitioners from related areas which make occasional usage of ML techniques, such as scientific computing.

Performance enhancement of programs is a wide subject including parallelization, and port between architectures and languages. Many tools and approaches exist outside the Python world, and beyond ML and DS. However, to deliver a consistent and organized view on the subject, we restrain our subject to cover the three main scenarios that could occur from a data scientist's point of view. Indeed, this is a partial and oriented view on the subject, leaving space for further explorations.

It is true that software development is a fast-paced domain, and that the best options today may be superseded by their competitors or new players only a couple of years after this work is published. However, we believe that the scenarios that back the structure of our study are general enough to remain true even as new tools are developed and existing tools evolve. In addition, our study serves to highlight a few basic facets of tools to which new or evolving tools can be fairly straightforwardly attached (e.g., related to task-based frameworks in Section 5.2 or drop-in numerical libraries in Section 4). Therefore, even as time goes by, we believe the insights we have delineated will provide useful guidance to practitioners for years to come.

As stated previously, when we delimited our search scope, we deliberately excluded Python interpreters from our study, as they are likely to interact with libraries mostly used in DS and ML domains. Yet, there are many contributions in this area that deeply affect vanilla Python efficiency: we briefly review them next.

6.3 Python Interpreters

CPython serves as the primary testbed for new features and language enhancements in Python. This is because it is maintained by the same community that designs the language. The performance challenges encountered in Python are directly associated with the CPython implementation. The work by Zhang et al. [86] shows potential performance improvements achievable through different optimization approaches in standard CPython interpreters. These optimizations include techniques like dispatch, branch prediction, and array-style access.

There are different implementations of the Python interpreter, offering developers the flexibility to replace the default CPython interpreter with alternative options. The advantage of these alternative interpreters is that they can enhance the speed and performance of Python programs without any code modification *a priori*. However, it is important to note that alternative interpreters may have certain limitations and drawbacks that need to be considered.

Alternative Python interpreters include PyPy [66], Pyston [48], Cinder [29], and IronPython [53]. However, not all of them offer complete coverage of the Python language. Additionally, some implementations are tied to specific Python versions. Cinder and Pyston, for example, support Python 3.8, IronPython supports Python 3.4, and PyPy supports Python 3.9 (which is relatively closer to the latest CPython versions). Historically, alternative implementations of Python have often lagged behind in terms of supporting the latest language versions and features.

Beyond being bound to Python 3.8, Pyston has no known compatibilities issues.[9] Pyston provides a performance gain estimated between 10% and 35% on reported benchmarks, despite some

[9] https://blog.pyston.org/2022/ (accessed May 24, 2022)

overheads observed on specific tasks such as JSON loading.[10] For some time, Pyston has been backed by Anaconda, a popular Python distribution tool. According to recent notes from one of the creators of Pyston, it is likely that their speedups will become fully integrated in a future CPython release.[11]

Cinder is a JIT interpreter implemented in C++ that includes several performance optimizations such as bytecode inline caching, eager evaluation of coroutines, and a method-at-a-time JIT. According to a report from their maintainers, it can provide a speedup of 1.5 to 4 on many Python performance benchmarks.[12] PyPy [66] is another alternative Python interpreter featuring a JIT compiler. According to benchmark results, with PyPy, the acceleration ratio can range from 0.21 to 4.8[13] with respect to CPython, with documented mixed results.[14] As already mentioned in Section 3.3, the widely used CPython interpreter does not include a built-in JIT compiler. Nonetheless, third-party libraries or tools, such as Pyston-lite (see Section 3.3.2), can be utilized to introduce JIT compilation into the Python workflow for performance optimization.

IronPython is an implementation of the Python programming language that runs on the .NET framework, providing seamless integration with .NET technologies and allowing the use of Python libraries. Its performance is comparable to CPython with variations depending on the specific task.[15] It is important to note that IronPython is primarily designed for Windows environments and does not support the importation of C extension modules, which limits its compatibility with certain libraries like NumPy and SciPy. As a result, some third-party libraries commonly used in DS and ML may not be available in IronPython.

A common problem with Python interpreters is that standard libraries—which may depend on other libraries—are not necessarily compliant outside the CPython implementation, and even so, they often require building shared libraries from the source. This may make it hard to validate the approach for each library and framework, and can be cumbersome for the average practitioner. The Python ecosystem is primarily built around CPython, which means that some community-supported projects, tools, and resources may not work seamlessly, with the added difficulty to keep up with the latest versions and language features.

7 CONCLUSION

Our article highlighted different approaches to enhance Python performance regarding three scenarios meant to cover most needs happening in the practice of DS and ML. Each scenario covers a peculiar stereotype of developer dealing with ML and DS tasks. They depict practitioner profiles that range from a very straightforward way of using Python (i.e., vanilla Python), by use of standard numerical libraries, up to the use of large integrated frameworks.

By answering our research question (Which approaches can be used to improve Python execution performance in the context of one of these three scenarios?), we have looked at the most relevant state-of-the-art approaches, following a narrative review principle. Each scenario calls for specific solutions which may be addressed by different kinds of techniques. For each scenario, we highlighted how given tools may help them deal with their task. We also highlighted the estimated complexity to set up those approaches, notably by the impact on the original code and in terms of the learning curve.

[10] https://www.phoronix.com/news/Pyston-2.1-vs-Python-3.8-3.9 (accessed May 24, 2022)
[11] https://blog.pyston.org/author/kmodzelewski (consulted in March 2023)
[12] https://github.com/facebookincubator/cinder
[13] https://speed.pypy.org/
[14] https://pybenchmarks.org/u64q/benchmark.php?test=all&lang=python3&lang2=pypy3&data=u64q
[15] https://www.python.org/about/success/resolver/

We showed that for pure Python code acceleration, the practitioners have a large choice depending on their level of confidence and control they want to have on the performance improvement. For simple and fast results, but not optimal, they may look at a diverse range of straightforward techniques, some even fully automatic, involving compiler directives, code decorators, or transpilers. Better performance can be obtained with semi-automatic approaches, but they require more involvement from the developer and a steeper learning curve for maximal gains.

In the case in which the codebase heavily relies on well-known numerical libraries, the most natural path is to investigate using drop-in libraries. Most of them mimic the API of the library they substitute to, so the learning curve is mild. However, for maximal gains, the practitioner must address subtleties such as memory movements between central and GPU memories.

In the third scenario, the practitioner is starting the development from scratch. Therefore, approaches surveyed in Section 5 are meant to be used right from the start of project development and put heavy constraints of code structure. This initial effort is traded with maximal gains in terms of performance and minimal surplus of effort if the driving principles of the frameworks are enforced.

We expect this work to give a good comprehensive view and guide practitioners in choosing among the plethora of existing tools. Our narrative structure and user scenarios were not only meant to facilitate document structuring, but also to provide a good starting point for such practitioners. All the relevant references are then available in one place for deeper inspection. In addition, all surveyed tools are reported in summary tables, one per user profile, emphasizing distinctive characteristics and providing usage metrics as to the date this article was written. Although we tried to be as comprehensive as possible, some features of the surveyed tools may not have been covered. As well, we did not run and quantitatively compare the performance of all the surveyed tools due to their number and diversity. It would be almost impossible to find a suitable common benchmark for any Python acceleration method and task-dedicated tool. We also expect that our work could help new tool designers who aim at enhancing Python performance to get an overview of the current state of the art.

REFERENCES

[1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Gregory S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian J. Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Józefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Gordon Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul A. Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda B. Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: Large-scale machine learning on heterogeneous distributed systems. *CoRR abs/1603.04467* (2016). http://arxiv.org/abs/1603.04467

[2] J. Akeret, L. Gamper, A. Amara, and A. Refregier. 2015. HOPE: A Python just-in-time compiler for astrophysical computations. *Astronomy and Computing* 10 (2015), 1–8. https://doi.org/10.1016/j.ascom.2014.12.001

[3] Nader Al Awar, Steven Zhu, George Biros, and Milos Gligoric. 2021. A performance portability framework for Python. In *Proceedings of the ACM International Conference on Supercomputing*. ACM, New York, NY, 467–478. https://doi.org/10.1145/3447818.3460376

[4] The Theano Development Team. 2016. Theano: A Python framework for fast computation of mathematical expressions. *arXiv e-prints arXiv:1605.02688* (2016).

[5] Juan D. Arcila-Moreno, Diego Alejandro Cifuentes Garcia, Francisco Jose Correa Zabala, Esteban Echeverri Jaramillo, Christian Trefftz, and Andres Felipe Zapata-Palacio. 2021. PyPacho: A Python library that implements parallel basic operations on GPUs. In *Proceedings of the IEEE Annual Information Technology, Electronics, and Mobile Communication Conference*. 229–238. https://doi.org/10.1109/IEMCON53756.2021.9623197

[6] Yadu Babuji, Anna Woodard, Zhuozhao Li, Daniel S. Katz, Ben Clifford, Rohan Kumar, Lukasz Lacinski, Ryan Chard, Justin M. Wozniak, Ian Foster, Michael Wilde, and Kyle Chard. 2019. Parsl: Pervasive parallel programming in Python. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*. 25–36.

[7] D. F. Bacon, S. L. Graham, and O. J. Sharp. 1994. Compiler transformations for high-performance computing. *ACM Computing Surveys* 26, 4 (1994), 345–420.

[8] C. Bastoul. 2004. Code generation in the polyhedral model is easier than you think. In *Proceedings of the International Conference on Parallel Architecture and Compilation Techniques*. 7–16. https://doi.org/10.1109/PACT.2004.1342537
[9] Michael Edward Bauer. 2014. *Legion: Programming Distributed Heterogeneous Architectures with Logical Regions*. Stanford University.
[10] Tal Ben-Nun, Johannes de Fine Licht, Alexandros Nikolaos Ziogas, Timo Schneider, and Torsten Hoefler. 2019. Stateful dataflow multigraphs: A data-centric model for performance portability on heterogeneous architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*.
[11] F. Z. Boito, E. C. Inacio, J. L. Bez, P. O. A. Navaux, M. A. R. Dantas, and Y. Denneulin. 2018. A checkpoint of research on parallel I/O for high-performance computing. *ACM Computing Surveys* 51, 2 (2018), Article 23, 35 pages.
[12] Uday Bondhugula, J. Ramanujam, and P. Sadayappan. 2007. *PLuTo: A Practical and Fully Automatic Polyhedral Parallelizer and Locality Optimizer*. Technical Report OSU-CISRC-10/07-TR70. The Ohio State University.
[13] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. JAX: Composable transformations of Python+NumPy programs. Retrieved September 9, 2023 from http://github.com/google/jax
[14] Maarten A. Breddels and Jovan Veljanoski. 2018. Vaex: Big data exploration in the era of Gaia. *Astronomy & Astrophysics* 618 (2018), A13.
[15] L. Bustio-Martínez, R. Cumplido, M. Letras, R. Hernández-León, C. Feregrino-Uribe, and J. Hernández-Palancar. 2021. FPGA/GPU-based acceleration for frequent itemsets mining: A comprehensive review. *ACM Computing Surveys* 54, 9 (2021), Article 179, 35 pages.
[16] Mateusz Bysiek, Mohamed Wahib, Aleksandr Drozd, and Satoshi Matsuoka. 2018. *Towards Portable High Performance in Python: Transpilation, High-Level IR, Code Transformations and Compiler Directives*. Technical Report 38. Tokyo Institute of Technology, National Institute of Advanced Industrial Science and Technology, RIKEN Center for Computational Science.
[17] Oscar Castro, Pierrick Bruneau, Jean-Sébastien Sottet, and Dario Torregrossa. 2022. Parallelization of data science tasks, an experimental overview. In *Proceedings of the International Conference on Computing and Pattern Recognition*. 483–490.
[18] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MXNet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274* (2015).
[19] The Keras Team. 2015. Keras. Retrieved September 9, 2023 from https://github.com/fchollet/keras
[20] Luis Pedro Coelho. 2017. Jug: Software for parallel reproducible computation in Python. *Journal of Open Research Software* 5, 1 (2017), 30.
[21] Ask Solem and Contributors. 2022. Celery—Distributed Task Queue. Retrieved September 9, 2023 from https://docs.celeryq.dev/en/stable/
[22] Shed Skin Contributors. 2022. Shed Skin: An Experimental (Restricted-Python)-to-C++ Compiler. Retrieved September 9, 2023 from https://shedskin.github.io/
[23] David M. Cooke and Francesc Alted. 2022. NumExpr: Fast Numerical Expression Evaluator for NumPy. Retrieved September 9, 2023 from https://numexpr.readthedocs.io/
[24] L. Dagum and R. Menon. 1998. OpenMP: An industry-standard API for shared-memory programming. *IEEE Computational Science and Engineering* 5, 1 (1998), 46–55. https://doi.org/10.1109/99.660313
[25] Lisandro Dalcín, Rodrigo Paz, and Mario Storti. 2005. MPI for Python. *Journal of Parallel and Distributed Computing* 65, 9 (2005), 1108–1115. https://doi.org/10.1016/j.jpdc.2005.03.010
[26] Dask Development Team. 2016. Dask: Library for Dynamic Task Scheduling. Retrieved September 9, 2023 from https://dask.org
[27] Datatable Contributors. 2022. Datatable: Python Library for Manipulating Tabular Data. Retrieved September 9, 2023 https://datatable.readthedocs.io/
[28] J. Dean and S. Ghemawat. 2008. MapReduce: Simplified data processing on large clusters. *Communications of the ACM* 51, 1 (2008), 107–113.
[29] Facebook. 2020. Cinder: Meta's Internal Performance-Oriented Production Version of CPython 3.8. Retrieved September 9, 2023 https://github.com/facebookincubator/cinder
[30] Juan J. Galvez, Karthik Senthil, and Laxmikant Kale. 2018. CharmPy: A Python parallel programming model. In *Proceedings of the IEEE International Conference on Cluster Computing*. 423–433. https://doi.org/10.1109/CLUSTER.2018.00059
[31] Serge Guelton, Pierrick Brunet, Mehdi Amini, Adrien Merlini, Xavier Corbillon, and Alan Raynaud. 2015. Pythran: Enabling static optimization of scientific Python programs. *Computational Science & Discovery* 8, 1 (2015), 014001. https://doi.org/10.1088/1749-4680/8/1/014001

[32] Panagiotis E. Hadjidoukas, Andrea Bartezzaghi, Florian Scheidegger, Roxana Istrate, Costas Bekas, and A. Cristiano I. Malossi. 2020. torcpy: Supporting task parallelism in Python. *SoftwareX* 12 (2020), 100517.

[33] Panagiotis E. Hadjidoukas, Evaggelos Lappas, and Vassilios V. Dimakopoulos. 2012. A runtime library for platform-independent task parallelism. In *Proceedings of the Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. 229–236. https://doi.org/10.1109/PDP.2012.89

[34] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. 2020. Array programming with NumPy. *Nature* 585, 7825 (2020), 357–362.

[35] Kay Hayen and Nuitka Contributors. 2022. Nuitka the Python Compiler. Retrieved September 9, 2023 from https://www.nuitka.net/index.html

[36] Yannick Hold-Geoffroy, Olivier Gagnon, and Marc Parizeau. 2014. Once you SCOOP, no need to fork. In *Proceedings of the Annual Conference on Extreme Science and Engineering Discovery Environment (XSEDE'14)*. ACM, New York, NY. https://doi.org/10.1145/2616498.2616565

[37] Facebook Incubator. 2017. Gloo. Retrieved September 9, 2023 from https://github.com/facebookincubator/gloo

[38] IPython Development Team and Enthought. 2022. DistArray: Think Globally, Act Locally. Retrieved September 9, 2023 from http://docs.enthought.com/distarray/

[39] Mohamed Ismail and G. Edward Suh. 2018. Quantitative overhead analysis for Python. In *Proceedings of the IEEE International Symposium on Workload Characterization*. 36–47.

[40] Dejice Jacob and Jeremy Singer. 2019. ALPyNA: Acceleration of loops in Python for novel architectures. In *Proceedings of the ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming (ARRAY'19)*. ACM, New York, NY, 25–34. https://doi.org/10.1145/3315454.3329956

[41] Laxmikant V. Kale and Sanjeev Krishnan. 1993. Charm++: A portable concurrent object oriented system based on C++. In *Proceedings of the 8th Annual Conference on Object-Oriented Programming Systems, Languages, and Applications*. 91–108.

[42] Mads R. B. Kristensen, Simon A. F. Lund, Troels Blum, Kenneth Skovhede, and Brian Vinter. 2013. Bohrium: Unmodified NumPy code on CPU, GPU, and cluster. In *Proceedings of the Workshop on Python for High Performance and Scientific Computing*.

[43] Mads Ruben Burgdorff Kristensen and Brian Vinter. 2010. Numerical Python for scalable architectures. In *Proceedings of the 4th Conference on Partitioned Global Address Space Programming Model (PGAS'10)*. ACM, New York, NY. https://doi.org/10.1145/2020373.2020388

[44] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. 2015. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM'15)*. ACM, New York, NY. https://doi.org/10.1145/2833157.2833162

[45] C. Lassner. 2015. Pymp. Retrieved September 9, 2023 from https://github.com/classner/pymp

[46] Timothy G. Mattson, Todd A. Anderson, and Giorgis Georgakoudis. 2021. PyOMP: Multithreaded parallel programming in Python. *Computing in Science & Engineering* 23, 6 (2021), 77–80. https://doi.org/10.1109/MCSE.2021.3128806

[47] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, D. B. Tsai, Manish Amde, Sean Owen, Doris Xin, Reynold Xin, Michael J. Frankin, Reza Zadeh, Matei Zaharia, and Ameet Talwalkar. 2016. MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research* 17, 1 (2016), 1235–1241.

[48] Kevin Modzelewski. 2022. Pyston. Retrieved September 9, 2023 from https://www.pyston.org/

[49] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A distributed framework for emerging AI applications. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. 561–577. https://www.usenix.org/conference/osdi18/presentation/moritz

[50] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A distributed framework for emerging AI applications. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. 561–577.

[51] John Magnus Morton, Kuba Kaszyk, Lu Li, Jiawen Sun, Christophe Dubach, Michel Steuwer, Murray Cole, and Michael F. P. O'Boyle. 2020. DelayRepay: Delayed execution for kernel fusion in Python. In *Proceedings of the ACM SIGPLAN International Symposium on Dynamic Languages (DLS'20)*. ACM, New York, NY, 43–56. https://doi.org/10.1145/3426422.3426980

[52] Stefan C. Müller, Gustavo Alonso, Adam Amara, and André Csillaghy. 2014. Pydron: Semi-automatic parallelization for multi-core and the cloud. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. 645–659. https://www.usenix.org/system/files/conference/osdi14/osdi14-paper-muller.pdf

[53] NET Foundation. 2023. IronPython. Retrieved September 9, 2023 from https://ironpython.net/

[54] Royud Nishino and Shohei Hido Crissman Loomis. 2017. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of the Conference on Neural Information Processing Systems*. 151.
[55] NVIDIA. 2022. RAPIDS: Open GPU Data Science. Retrieved September 9, 2023 from https://rapids.ai/
[56] The Pandas Development Team. 2022. pandas-dev/pandas: Pandas. Retrieved September 9, 2023 from https://github.com/pandas-dev/pandas
[57] Parallelpython.com. 2022. Parallel Python. Retrieved September 9, 2023 from https://www.parallelpython.com/
[58] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32 (2019), 8024–8035.
[59] N. Paulino, J. C. Ferreira, and J. M. P. Cardoso. 2020. Improving performance and energy consumption in embedded systems via binary acceleration: A survey. *ACM Computing Surveys* 53, 1 (2020), Article 6, 36 pages.
[60] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
[61] Devin Petersohn, Stephen Macke, Doris Xin, William Ma, Doris Lee, Xiangxi Mo, Joseph E. Gonzalez, Joseph M. Hellerstein, Anthony D. Joseph, and Aditya Parameswaran. 2020. Towards scalable dataframe systems. *Proceedings of the VLDB Endowment* 13 (2020), 2033–2046. https://doi.org/10.14778/3407790.3407807
[62] Polars. 2022. Polars Lightning-Fast DataFrame Library for Rust and Python. Retrieved September 9, 2023 from https://github.com/pola-rs/polars
[63] Pyjion. 2023. Pyjion: A Drop-in JIT Compiler for Python. Retrieved September 9, 2023 from https://www.trypyjion.com
[64] PyMC. 2020. Aesara. Retrieved September 9, 2023 from https://github.com/aesara-devs/aesara
[65] PYPL. 2022. PopularitY of Programming Language. Retrieved September 9, 2023 from https://pypl.github.io/PYPL.html
[66] PyPy. 2022. How Fast is PyPy3.9? Retrieved September 9, 2023 from https://speed.pypy.org/
[67] Cristian Ramon-Cortes, Ramon Amela, Jorge Ejarque, Philippe Clauss, and Rosa M. Badia. 2020. AutoParallel: Automatic parallelisation and distributed execution of affine loop nests in Python. *International Journal of High Performance Computing Applications* 34, 6 (2020), 659–675. https://doi.org/10.1177/1094342020937050
[68] NVIDIA Rapids. 2022. cuDF—GPU DataFrame Library. Retrieved September 9, 2023 from https://github.com/rapidsai/cudf
[69] J. A. Rico-Gallego, J. C. Díaz-Martín, Ravi R. Manumachu, and A. L. Lastovetsky. 2019. A survey of communication performance models for high-performance computing. *ACM Computing Surveys* 51, 6 (2019), Article 126, 36 pages.
[70] Dale Roberts. 2022. PyPar. Retrieved September 9, 2023 from https://github.com/daleroberts/pypar
[71] Matthew Rocklin. 2015. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the Python in Science Conference*, Vol. 130. 136.
[72] Cyrille Rossant, Bertrand Fontaine, and Dan F. M. Goodman. 2013. Playdoh: A lightweight Python library for distributed computing and optimisation. *Journal of Computational Science* 4, 5 (2013), 352–359. https://doi.org/10.1016/j.jocs.2011.06.002
[73] Karl Rupp. 2022. PyViennaCL. Retrieved September 9, 2023 from http://viennacl.sourceforge.net/.
[74] Amit Sabne. 2020. XLA: Compiling Machine Learning for Peak Performance. Retrieved September 9, 2023 from https://research.google/pubs/pub50530/
[75] Alexander Sergeev and Mike Del Balso. 2018. Horovod: Fast and easy distributed deep learning in TensorFlow. *arXiv preprint arXiv:1802.05799* (2018).
[76] Elliott Slaughter and Alex Aiken. 2019. Pygion: Flexible, scalable task-based parallelism with Python. In *Proceedings of the IEEE/ACM Parallel Applications Workshop, Alternatives to MPI (PAW-ATM'19)*. IEEE, Los Alamitos, CA, 58–72.
[77] Leonhard Spiegelberg, Rahul Yesantharao, Malte Schwarzkopf, and Tim Kraska. 2021. *Tuplex: Data Science in Python at Native Code Speed*. ACM, New York, NY, 1718–1731. https://doi.org/10.1145/3448016.3457244
[78] Theo Steininger, Maksim Greiner, Frederik Beaujean, and Torsten Enßlin. 2016. D2O: A distributed data object for parallel high-performance computing in Python. *Journal of Big Data* 3, 1 (2016), 1–34.
[79] SymPy Development Team. 2022. SymPy: Python Library for Symbolic Mathematics. Retrieved September 9, 2023 from https://www.sympy.org/
[80] IPython Development Team. 2022. Using IPython for Parallel Computing. Retrieved September 9, 2023 from https://ipyparallel.readthedocs.io/
[81] Enric Tejedor and Rosa M. Badia. 2008. COMP Superscalar: Bringing GRID Superscalar and GCM together. In *Proceedings of the IEEE International Symposium on Cluster Computing and the Grid*. 185–193. https://doi.org/10.1109/CCGRID.2008.104

[82] Enric Tejedor, Yolanda Becerra, Guillem Alomar, Anna Queralt, Rosa M. Badia, Jordi Torres, Toni Cortes, and Jesús Labarta. 2017. PyCOMPSs: Parallel computational workflows in Python. *International Journal of High Performance Computing Applications* 31, 1 (2017), 66–82. https://doi.org/10.1177/1094342015594678

[83] TIOBE. 2022. TIOBE Index. Retrieved September 9, 2023 from https://www.tiobe.com/tiobe-index/

[84] Christian R. Trott, Damien Lebrun-Grandié, Daniel Arndt, Jan Ciesko, Vinh Dang, Nathan Ellingwood, Rahulkumar Gayatri, Evan Harvey, Daisy S. Hollman, Dan Ibanez, Nevin Liber, Jonathan Madsen, Jeff Miles, David Poliakoff, Amy Powell, Sivasankaran Rajamanickam, Mikael Simberg, Dan Sunderland, Bruno Turcksin, and Jeremiah Wilke. 2022. Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems* 33, 4 (2022), 805–817. https://doi.org/10.1109/TPDS.2021.3097283

[85] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. 2020. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods* 17 (2020), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[86] Qiang Zhang, Lei Xu, Xiangyu Zhang, and Baowen Xu. 2022. Quantifying the interpretation overhead of Python. *Science of Computer Programming* 215 (2022), 102759.

[87] Javier Álvarez Cid-Fuentes, Salvi Solà, Pol Álvarez, Alfred Castro-Ginard, and Rosa M. Badia. 2019. dislib: Large scale high performance machine learning in Python. In *Proceedings of the 15th International Conference on eScience*. 96–105.

Received 28 October 2022; revised 1 July 2023; accepted 21 August 2023